Universidade de São Paulo - ICMC/EESC - Curso de Eng. de Computação
SSC0903 - Computação de Alto Desempenho (2026/1) – Trab01

Nomes dos integrantes deste grupo que resolveram a avaliação:

André Vargas Villalba Codorniz
João Pedro Machado Medeiros
Leonardo Hannas de Carvalho Santos
Lucas Augusto Moreira Barros
Lucas Kimo Costa
Samuel Rubens Souza Oliveira

Resposta para a Q02:

Q2 - Análise de Desempenho

1 Configuração dos Experimentos

Os experimentos foram realizados com o objetivo de avaliar o desempenho da solução paralela em comparação com a versão sequencial, considerando diferentes cargas de trabalho e números de threads.

Foram utilizados os seguintes parâmetros:

- Número de regiões (R)
- Número de cidades por região (C)
- Número de alunos por cidade (A)
- Número de notas por aluno (N)

A metodologia experimental buscou conciliar o contexto geográfico do problema com a análise de desempenho em diferentes níveis de paralelismo. Para garantir reprodutibilidade, foi utilizada uma semente pseudoaleatória fixa igual a 7 em todos os testes, variando-se os parâmetros R, C, A, N e T.

Como o problema considera alunos distribuídos pelas regiões brasileiras, adotou-se $R \leq 5$, correspondendo às cinco macrorregiões do país. Os parâmetros C, A e N foram interpretados, respectivamente, como número de cidades por região, alunos por cidade e notas por aluno.

O número de threads foi variado em $T = 1, 2, 4, 8$, permitindo avaliar desde o caso sem paralelismo efetivo até cenários com maior grau de concorrência.

Cada configuração foi executada 5 vezes nas versões sequencial e paralela, sendo utilizados os tempos médios para o cálculo das métricas de desempenho. O tempo de execução foi medido desconsiderando operações de entrada e saída.

Todos os experimentos foram realizados com código compilado utilizando `gcc` com otimização `-O3`, garantindo melhor aproveitamento de cache, otimizações de loop e possíveis vetorizações:

```
gcc studentsseq.c -O3 -o seq -lm -fopenmp
gcc studentspar.c -O3 -o par -lm -fopenmp
```

A mesma configuração de compilação foi utilizada para ambas as versões, assegurando comparabilidade direta entre os resultados.

2 Métricas Utilizadas

O speedup foi calculado como:

$$S = \frac{T_{seq}}{T_{par}}$$

A eficiência foi calculada como:

$$E = \frac{S}{T}$$

em que T_{seq} representa o tempo médio da execução sequencial, T_{par} representa o tempo médio da execução paralela e T representa o número de threads utilizado no caso de teste.

3 Definição e Justificativa dos Casos de Teste

Na visão do grupo, os dados testados precisaram atender a duas demandas:

1. devem ser capazes de provar a diferença do poder computacional entre o código paralelo e o sequencial;
2. devem manter coerência demográfica com o enunciado, ou seja, uma simulação de avaliações de alunos do 9º ano em cidades distribuídas pelas regiões brasileiras.

3.1 Primeira bateria de testes (casos 1 a 20)

A primeira bateria avaliou variações graduais dos parâmetros (R, C, A, N) com diferentes números de threads, considerando cargas de trabalho pequenas. Esperava-se observar o comportamento inicial do paralelismo, porém muitos tempos medidos foram extremamente baixos (frequentemente $0.000\,s$), o que prejudicou a análise de desempenho. Nesses casos, o overhead do paralelismo e a resolução da medição tiveram impacto significativo, tornando métricas como speedup pouco confiáveis.

3.2 Segunda bateria de testes (casos 21 a 36)

A segunda bateria utilizou cargas mais elevadas, com o objetivo de obter medições mais estáveis e representativas. Nessa etapa, esperava-se observar ganhos mais claros com o aumento do número de threads, o que de fato ocorreu. Os resultados mostraram crescimento consistente do speedup e redução do tempo de execução, tornando essa bateria a principal base para a análise quantitativa.

3.3 Terceira bateria de testes (casos 37 a 48)

A terceira bateria considerou cargas ainda mais elevadas e adotou uma modelagem diferente das duas primeiras baterias: em vez de representar apenas as cinco macrorregiões brasileiras, o parâmetro R passou a representar os estados do Brasil. Assim, foi utilizado $R = 26$, considerando os 26 estados brasileiros como unidades independentes de processamento. Essa escolha aumentou significativamente a quantidade de blocos independentes no problema, favorecendo a exploração do paralelismo em diferentes níveis do algoritmo e permitindo avaliar a escalabilidade da solução sob condições mais intensivas.

Para manter coerência geográfica e, ao mesmo tempo, aumentar a carga computacional, optou-se por utilizar um número relativamente pequeno de cidades por estado ($C = 10, 12, 15$), mas com uma quantidade elevada de alunos por cidade ($A = 15\,000, 20\,000, 25\,000$) e mais notas por aluno ($N = 12$ ou 15). Essa escolha representa uma amostra agregada de grandes centros educacionais ou conjuntos

de escolas por estado, evitando assumir que todos os estados teriam centenas de municípios avaliados. Computacionalmente, essa configuração aumentou bastante o volume de dados processados, reduziu o impacto relativo do overhead de paralelização e produziu tempos mais expressivos. De fato, os ganhos de desempenho foram mais consistentes nessa bateria, tornando os casos 37 a 48 especialmente relevantes para avaliar o comportamento da solução paralela em cenários de maior intensidade computacional.

4 Resultados

As Tabelas 1, 2 e 3 apresentam os tempos médios, speedup e eficiência obtidos para cada bateria de testes.

Cada valor apresentado foi calculado a partir da média de 5 execuções da versão sequencial e 5 execuções da versão paralela.

A Tabela 1 corresponde à primeira bateria (casos 1 a 20), com cargas menores e caráter exploratório. A Tabela 2 apresenta a segunda bateria (casos 21 a 36), com cargas mais elevadas e resultados mais estáveis. Por fim, a Tabela 3 contém a terceira bateria (casos 37 a 48), com os cenários mais volumosos da análise.

Tabela 1: Resultados médios para os casos 1 a 20.

Caso	Parâmetros (R, C, A, N, T)	$\bar{T}_{seq}$ (s)	$\bar{T}_{par}$ (s)	Speedup	Eficiência
1	(1,2,30,4,1)	0.000000	0.000000	nan	0.000000
2	(1,2,30,4,2)	0.000000	0.000000	nan	0.000000
3	(1,2,30,4,4)	0.000000	0.000000	nan	0.000000
4	(1,2,30,4,8)	0.000000	0.000800	0.000000	0.000000
5	(2,5,80,6,1)	0.000000	0.000000	nan	0.000000
6	(2,5,80,6,2)	0.000000	0.000000	nan	0.000000
7	(2,5,80,6,4)	0.000000	0.000000	nan	0.000000
8	(2,5,80,6,8)	0.000000	0.000000	nan	0.000000
9	(3,10,150,8,1)	0.001000	0.001000	1.000000	1.000000
10	(3,10,150,8,2)	0.001000	0.000000	nan	0.000000
11	(3,10,150,8,4)	0.001000	0.000000	nan	0.000000
12	(3,10,150,8,8)	0.001000	0.001000	1.000000	0.125000
13	(5,20,300,8,1)	0.004000	0.004000	1.000000	1.000000
14	(5,20,300,8,2)	0.004000	0.003000	1.333333	0.666667
15	(5,20,300,8,4)	0.004000	0.002000	2.000000	0.500000
16	(5,20,300,8,8)	0.004000	0.003000	1.333333	0.166667
17	(5,40,800,10,1)	0.024000	0.024400	0.983607	0.983607
18	(5,40,800,10,2)	0.024000	0.016000	1.500000	0.750000
19	(5,40,800,10,4)	0.024200	0.012200	1.983607	0.495902
20	(5,40,800,10,8)	0.024000	0.011600	2.068966	0.258621

Tabela 2: Resultados médios para os casos 21 a 36.

Caso	Parâmetros (R, C, A, N, T)	$\bar{T}_{seq}$ (s)	$\bar{T}_{par}$ (s)	Speedup	Eficiência
21	(5,50,2000,10,1)	0.078600	0.079200	0.992424	0.992424
22	(5,50,2000,10,2)	0.078800	0.052200	1.509579	0.754790
23	(5,50,2000,10,4)	0.078000	0.039000	2.000000	0.500000
24	(5,50,2000,10,8)	0.078400	0.033800	2.319527	0.289941
25	(5,60,3000,10,1)	0.145800	0.147000	0.991837	0.991837
26	(5,60,3000,10,2)	0.145000	0.095200	1.523109	0.761555
27	(5,60,3000,10,4)	0.145200	0.070400	2.062500	0.515625
28	(5,60,3000,10,8)	0.145600	0.060800	2.394737	0.299342
29	(5,80,4000,12,1)	0.267800	0.269000	0.995539	0.995539
30	(5,80,4000,12,2)	0.267200	0.174800	1.528604	0.764302
31	(5,80,4000,12,4)	0.269600	0.129600	2.080247	0.520062
32	(5,80,4000,12,8)	0.268800	0.110600	2.430380	0.303797
33	(5,100,5000,12,1)	0.424600	0.430600	0.986066	0.986066
34	(5,100,5000,12,2)	0.424200	0.278000	1.525899	0.762949
35	(5,100,5000,12,4)	0.425600	0.214600	1.983225	0.495806
36	(5,100,5000,12,8)	0.429600	0.178200	2.410774	0.301347

Tabela 3: Resultados médios para os casos 37 a 48.

Caso	Parâmetros (R, C, A, N, T)	$\bar{T}_{seq}$ (s)	$\bar{T}_{par}$ (s)	Speedup	Eficiência
37	(26,10,15000,12,1)	0.671200	0.678600	0.989095	0.989095
38	(26,10,15000,12,2)	0.669400	0.429200	1.559646	0.779823
39	(26,10,15000,12,4)	0.668400	0.313400	2.132738	0.533184
40	(26,10,15000,12,8)	0.668800	0.274600	2.435543	0.304443
41	(26,12,20000,15,1)	1.098800	1.117800	0.983002	0.983002
42	(26,12,20000,15,2)	1.099200	0.694800	1.582038	0.791019
43	(26,12,20000,15,4)	1.100200	0.505200	2.177751	0.544438
44	(26,12,20000,15,8)	1.095800	0.447000	2.451454	0.306432
45	(26,15,25000,15,1)	1.754400	1.776400	0.987615	0.987615
46	(26,15,25000,15,2)	1.751800	1.114800	1.571403	0.785702
47	(26,15,25000,15,4)	1.746800	0.808600	2.160277	0.540069
48	(26,15,25000,15,8)	1.751400	0.716800	2.443359	0.305420

5 Gráficos

Nesta seção, são apresentados gráficos com o objetivo de complementar a análise numérica das tabelas, permitindo uma visualização mais clara do comportamento da versão paralela em diferentes cenários de carga e número de threads.

Os gráficos foram construídos a partir dos tempos médios obtidos em cada caso de teste (média de 5 execuções), considerando principalmente as baterias 21–36 e 37–48, que apresentam cargas de trabalho suficientemente grandes para gerar medições mais estáveis e representativas.

Inicialmente, são analisadas as métricas derivadas de desempenho: speedup e eficiência. Por fim, apresenta-se a comparação direta entre os tempos médios das versões sequencial e paralela.

5.1 Speedup em função do número de threads

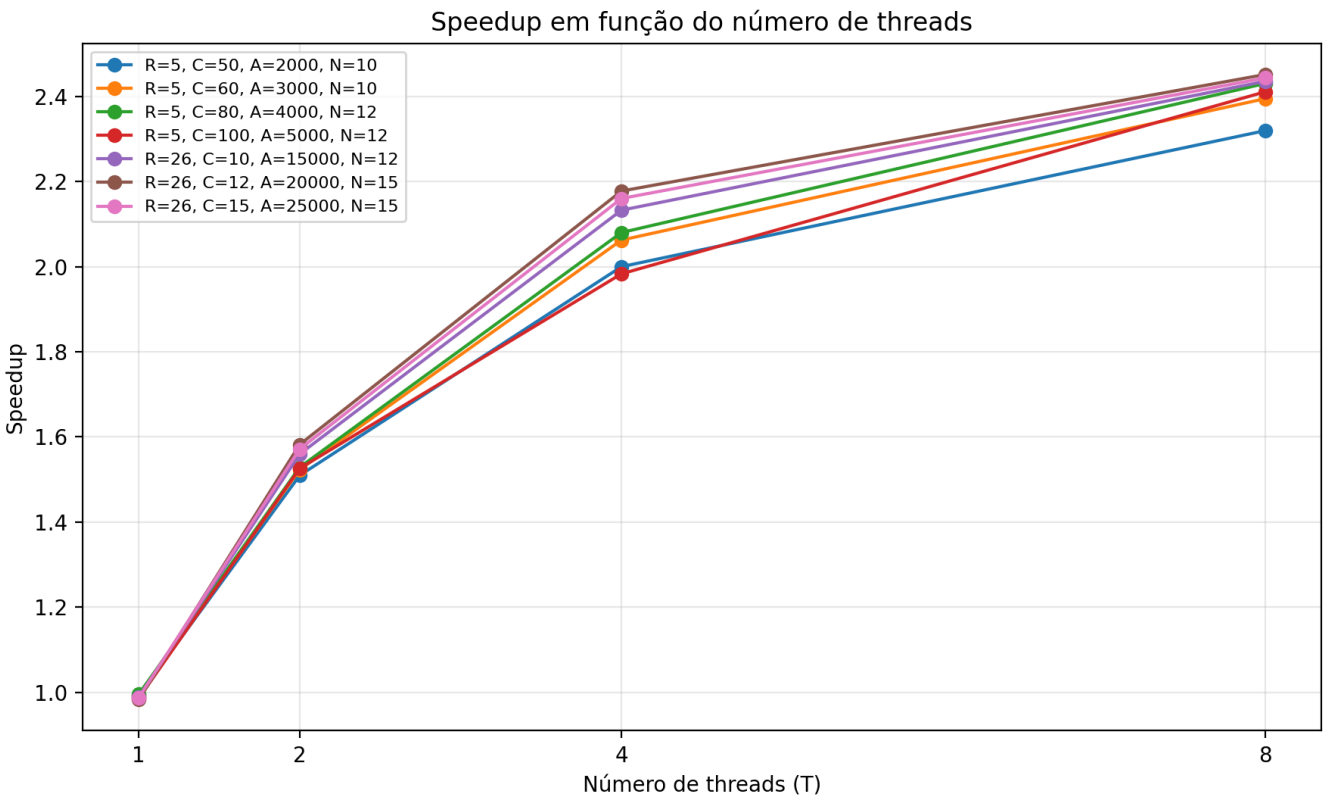

Figura 1: Speedup em função do número de threads

O gráfico de speedup evidencia o ganho relativo de desempenho obtido com o aumento do número de threads. Observa-se que o speedup cresce consistentemente com o aumento de T, especialmente nas baterias com maior volume de dados (casos 21–48).

Entretanto, esse crescimento não é linear, indicando limitações típicas do paralelismo, como overhead de criação e sincronização de threads, além de possíveis gargalos de memória e partes não paralelizáveis do algoritmo.

5.2 Eficiência em função do número de threads

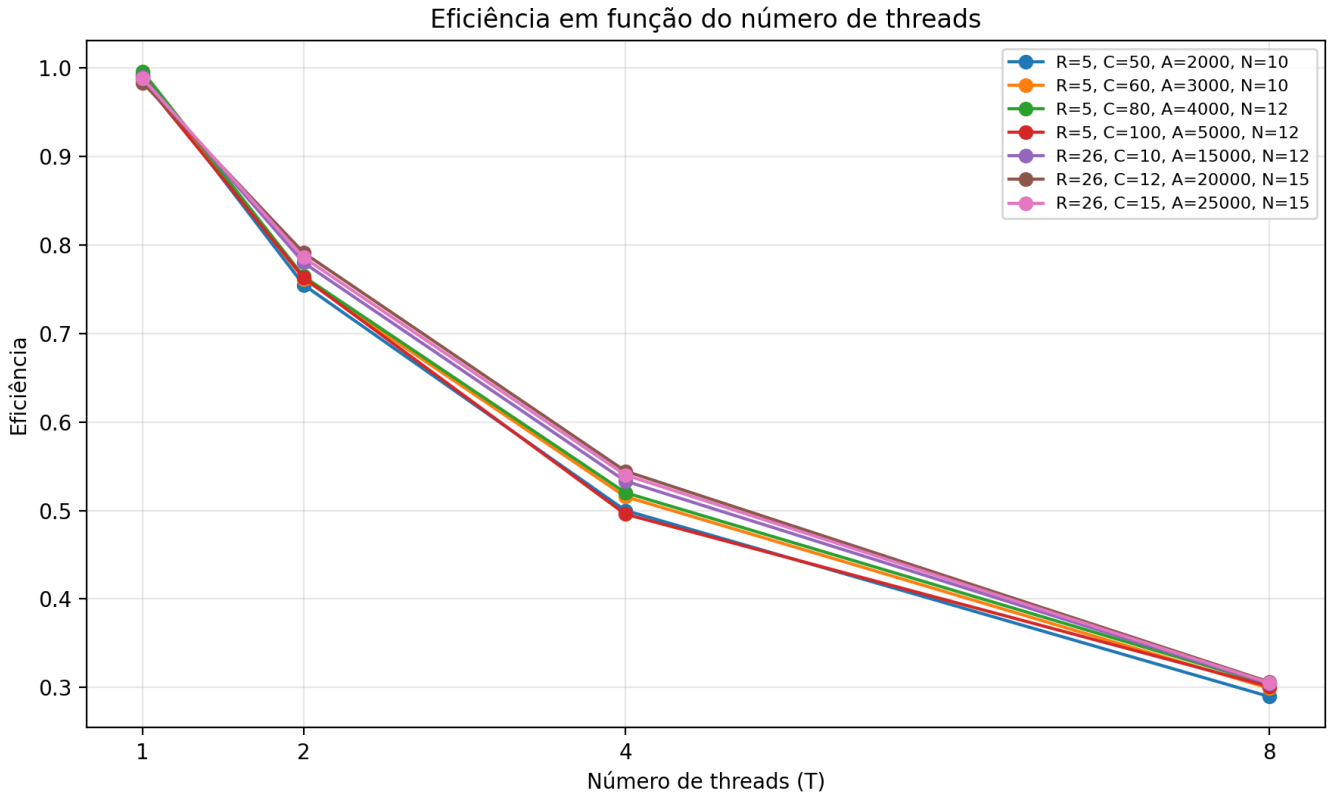

Figura 2: Eficiência em função do número de threads

O gráfico de eficiência mostra como o ganho de desempenho por thread se comporta à medida que o paralelismo aumenta. Observa-se que a eficiência diminui conforme o número de threads cresce, comportamento esperado em sistemas paralelos.

Apesar disso, nos casos de maior carga (principalmente na terceira bateria), a eficiência se mantém em níveis razoáveis, indicando que o aumento do volume de dados contribui para um melhor aproveitamento do paralelismo.

5.3 Comparação de tempos de execução

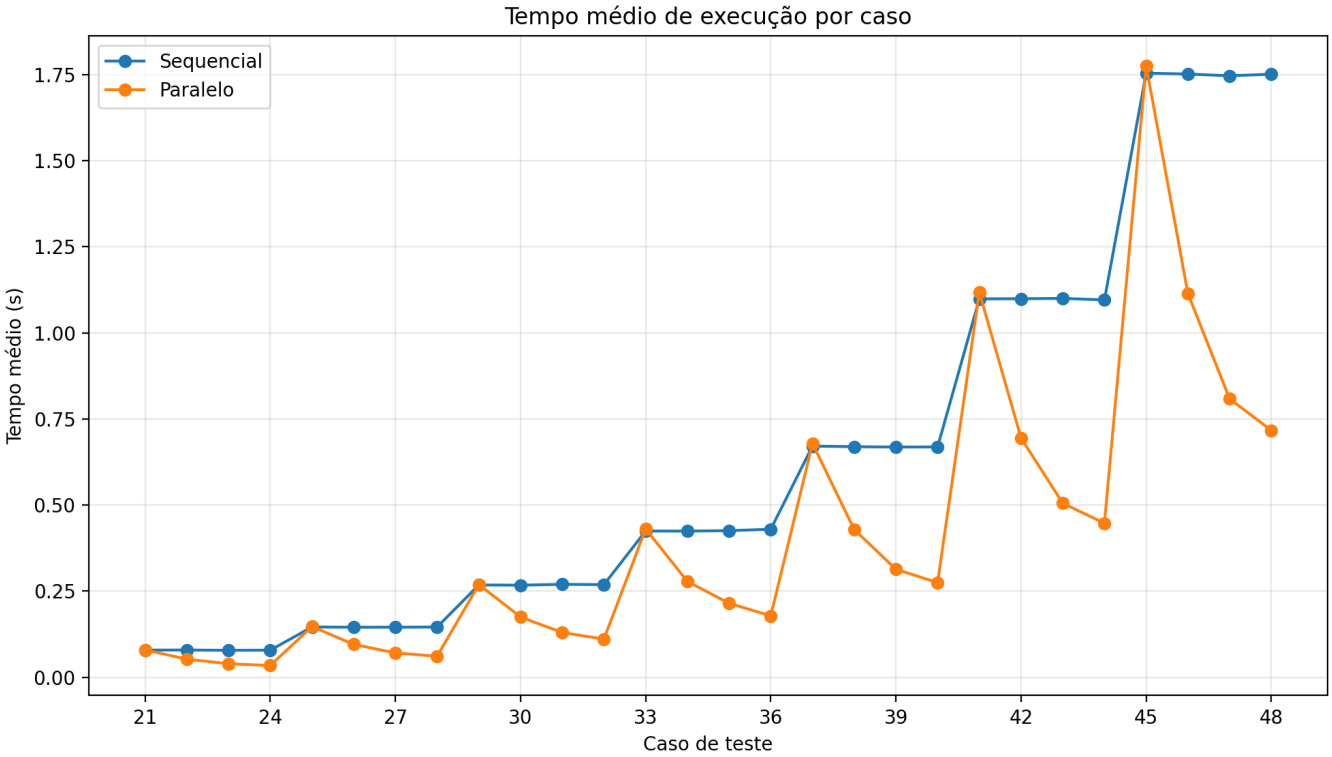

Figura 3: Tempo médio sequencial vs. paralelo para os casos 21 a 48

O gráfico acima mostra a evolução dos tempos médios de execução para as versões sequencial e paralela ao longo dos casos de teste mais relevantes. Observa-se que, à medida que a carga de trabalho aumenta (especialmente nas baterias 21–36 e 37–48), a versão paralela passa a apresentar ganhos mais expressivos em relação à versão sequencial.

Além disso, nota-se que, para $T = 1$, a versão paralela apresenta desempenho semelhante ou ligeiramente inferior ao sequencial, o que é esperado devido ao overhead da região paralela sem ganho efetivo de paralelismo.

6 Análise dos Resultados

Observa-se que o aumento do número de threads reduz o tempo de execução da versão paralela, especialmente para cargas maiores. O speedup cresce com o número de threads, mas não de forma linear, devido ao overhead de criação e sincronização e à competição por recursos compartilhados. Como consequência, a eficiência diminui à medida que T aumenta. Para cargas pequenas, o paralelismo traz ganhos limitados e pode até apresentar desempenho semelhante ou inferior ao sequencial.

Na primeira bateria (casos 1 a 20), muitos tempos medidos foram extremamente baixos (próximos de $0.000, s$), tornando speedup e eficiência pouco informativos. Esses resultados evidenciam que, para problemas muito pequenos, o overhead do paralelismo pode anular qualquer ganho.

A segunda bateria (casos 21 a 36) apresentou resultados mais robustos. Para $T = 1$, a versão paralela teve desempenho semelhante ou ligeiramente inferior ao sequencial. Com $T = 2$, os speedups ficaram em torno de 1.50–1.53; com $T = 4$, entre 2.00–2.08; e com $T = 8$, entre 2.32–2.43. Nesse último caso, a eficiência caiu para cerca de 0.29–0.30, indicando retorno marginal decrescente por thread.

O aumento de C, A e N nessa bateria mostrou que cargas maiores produzem medições mais confiáveis e favorecem o paralelismo, ao aumentar o volume de trabalho independente.

Diferenças iniciais entre as versões foram corrigidas (comparação em ponto flutuante, reduções e precisão dos acumuladores), garantindo saídas idênticas entre as versões sequencial e paralela.

A terceira bateria (casos 37 a 48) confirmou os padrões anteriores em cargas mais volumosas. Com $T = 1$, o desempenho permaneceu próximo ao sequencial (speedup ≈ 1). Para $T = 2$, os speedups ficaram entre 1.56–1.58; para $T = 4$, entre 2.13–2.18; e para $T = 8$, entre 2.43–2.45, com eficiência novamente próxima de 0.30.

Essa bateria também aumentou a granularidade do problema ao considerar $R = 26$, ampliando a quantidade de unidades independentes e tornando os tempos mais representativos. Os resultados mostram que, embora o speedup não seja linear, há redução consistente do tempo de execução em cenários de grande volume.

Assim, os resultados confirmam dois pontos clássicos: o paralelismo não compensa para problemas pequenos e, para cargas maiores, o speedup cresce de forma consistente, embora com queda de eficiência conforme o número de threads aumenta.

7 Conclusão

A solução paralela demonstrou ganhos significativos de desempenho para cargas de trabalho maiores, evidenciando a eficácia do uso de OpenMP. Por outro lado, os resultados também evidenciam limitações típicas do paralelismo, como a redução da eficiência com o aumento do número de threads, devido a overhead e competição por recursos.

Em síntese, os experimentos mostraram que a versão paralela manteve consistência com a versão sequencial e apresentou melhoria relevante de desempenho nos cenários de maior carga. A primeira bateria teve caráter exploratório e evidenciou que, para entradas pequenas, o paralelismo não é vantajoso. Já a segunda bateria permitiu uma avaliação mais estável e confiável de speedup e eficiência, deixando claro o ganho progressivo com o aumento do número de threads, ainda que não linear.

A terceira bateria reforçou esses resultados ao adotar uma modelagem estadual ($R = 26$) e cargas significativamente maiores. Nesses cenários, observaram-se speedups estáveis, principalmente para $T = 2$, $T = 4$ e $T = 8$, indicando melhor aproveitamento do paralelismo em problemas mais volumosos.

Dessa forma, conclui-se que a solução paralela é particularmente eficaz em cenários de grande escala, nos quais o volume de dados é suficiente para compensar os custos de gerenciamento das threads, embora a eficiência tenda a diminuir à medida que o nível de paralelismo aumenta.

8 Passo a passo para compilação e execução no cluster

1. Acessar o cluster utilizando as credenciais fornecidas:

```
ssh ssc903-ta-g02@andromeda.lasdpc.icmc.usp.br -p 2270
```

2. Navegar até o diretório do projeto:

```
cd ~/teste_no_cluster
```

3. Compilar a versão sequencial:

```
gcc studentsseq.c -O3 -o seq -lm -fopenmp
```

4. Compilar a versão paralela:

```
gcc studentspar.c -O3 -o par -lm -fopenmp
```

5. Executar a versão sequencial com um arquivo de entrada:

```
./seq caso21_r5_c50_a2000_n10_t1.txt
```

6. Executar a versão paralela com o mesmo arquivo:

```
./par caso21_r5_c50_a2000_n10_t1.txt
```

7. Executar um caso de maior carga:

```
./seq caso48_r26_c15_a25000_n15_t8.txt
./par caso48_r26_c15_a25000_n15_t8.txt
```

8. Comparar diretamente as saídas:

```
diff <(./seq caso21_r5_c50_a2000_n10_t1.txt) \
     <(./par caso21_r5_c50_a2000_n10_t1.txt)
```

   Caso não haja saída, os resultados são idênticos.

9. Comparar apenas os tempos de execução:

```
diff <(./seq caso21_r5_c50_a2000_n10_t1.txt | grep "Tempo") \
     <(./par caso21_r5_c50_a2000_n10_t1.txt | grep "Tempo")
```

   Nesse caso, espera-se diferença apenas nos valores de tempo.

Observação: os arquivos de entrada seguem o formato $(R, C, A, N, T, seed)$. O tempo exibido ao final corresponde ao tempo de execução em segundos, desconsiderando operações de entrada e saída.